Demo 1 Test Notes

Monday, 02 March 2026 20:49

Versions Used during Demo 1

PDD_v03

PDD_v02

PDD_v01

Components that were Implemented during Demo 1

- Reverse-Polarity diode
- L78C5 5V regulator
- LD1117V33 3.3V regulator
- LEDs
- Buttons
- Temperature Sensor

Software Implementation during Demo 1

- UART Student number
- LED flashing
- Button-toggling
- Temperature sensor
- Debouncing

Key Notes from Lecture 1 Slides

Lecture 3

Lecture 2

Lecture 1

What is a decoupling/bypass capacitor

A small capacitor placed between the power line and GND to "clean up" power signal, i.e.:

- Smooths out fluctuation
- Eliminates high frequency noise
- Protect IC from unstable power

What is Dropout Voltage

The minimum difference between V_{in} and V_{out} at which the regulator can still maintain the correct output V

Reverse-Polarity Diode

Monday, 02 March 2026 21:12

1N4007
DIODE

☑ Purpose of the Reverse-Polarity Diode

A reverse-polarity diode is added so that **if the user accidentally connects the power supply backwards**, the circuit does **not** receive negative voltage.

✓ What happens without the diode

If the battery or power supply is connected incorrectly (e.g., +9 V and GND swapped):

- negative voltage would be applied to the 7805 regulator
- ICs on the board would be reverse-biased
- electrolytic capacitors may be damaged
- components connected to the 5 V rail may fail instantly

Many components are not protected against reverse voltage, so the damage is often permanent.

✓ What the diode actually does in your circuit

In your schematic, **D1 (1N4007)** is placed in series with the input. It acts as a **one-way valve**:

- If the power is connected correctly → diode conducts normally → the regulator receives power.
- If the power is connected backwards → the diode becomes reverse-biased → **no current flows** → the rest of the circuit sees **0** V instead of a negative voltage.

So instead of destroying your board, the circuit simply **does not turn on**—which is safe.

✓ Additional benefit

Series reverse-polarity diodes also protect against:

- accidentally plugging in a center-negative barrel jack
- connecting a bench supply incorrectly
- automotive wiring mistakes

L78C5 5V regulator

Monday, 02 March 2026 20:54

5V_Regulator_L78C5

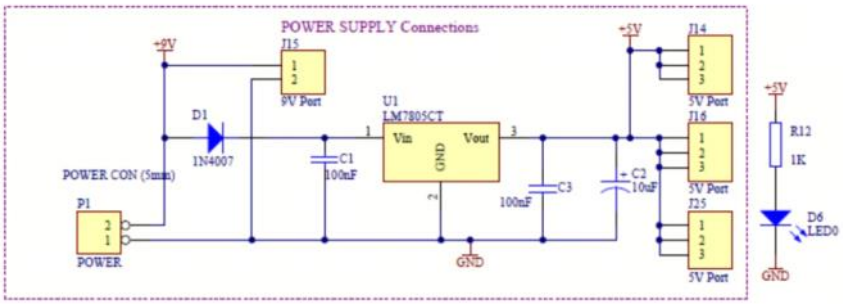


Figure 3: 5 V Power supply circuit diagram

What is the Purpose of C1 and C3? Why is C2 redundant?

C1 (100 nF) — Input decoupling capacitor

Purpose:
Placed *as close as possible* to the 7805 input pin.
It provides **high-frequency bypassing** to ground, stabilizing the regulator by handling fast transient currents **before** they reach the 7805.
Why 100 nF?
Small ceramic caps have low ESR and handle high-frequency noise extremely well.

C3 (100 nF) — Output decoupling capacitor

Purpose:
Also a **high-frequency bypass** cap, but on the output side.
This keeps the 7805 stable by preventing oscillation and giving it a clean, low-impedance load at high frequencies.
Why 100 nF?
Again: excellent for high-frequency stability.

C2 (10 µF) — Bulk output capacitor

Purpose:
This is a larger capacitor meant for **low-frequency smoothing** and providing short bursts of extra current when the load changes suddenly (e.g., microcontroller waking up, LEDs switching on).
So far: **C2 is useful.**

So why do people say C2 is "redundant"?

Because **the 7805 datasheet already requires only one output capacitor**, typically:

- **0.1 µF (for high-frequency stability)**
- optionally **1 µF to 10 µF (for load transients)**

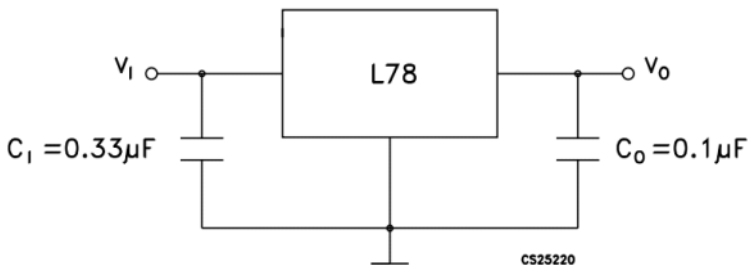
But your circuit **already has C3 (100 nF)** for stability.
If your load is light and steady, the **extra electrolytic (C2 = 10 µF)** doesn't do much and can be seen as optional.
In other words:

✓ C3 is required

✓ C2 is optional (nice to have, but not strictly required)

C2 *isn't harmful* — it just isn't essential unless your 5V load draws sudden or pulsed currents.

Figure 4. Application circuits



✓ The datasheet calls for 0.33 µF on the input, but

Your **100** nF ceramic C1 performs the same HF function *better* than the old 0.33 *µF tantalum/mylar did.*
Modern designs often replace the original 0.33 µF with **100** nF + bulk capacitance upstream.

✓ C3 (100 nF) matches the datasheet's 0.1 µF required output bypass

✓ C2 (10 µF) is optional bulk storage

It improves transient performance but is **not required for regulator stability**, which is why many people call it "redundant".

Is a Heat-Sink Necessary For the Regulator

It depends on how much power your 7805 is dissipating.

You only need a heat sink if the regulator is dissipating **more heat than the TO-220 package can safely handle without one.**

Below is the clear way to decide.

☑ 1. The key formula: how much heat does your 7805 produce?

Linear regulators turn the voltage drop into heat: $P_{\text{dissipated}} = (V_{\text{in}} - V_{\text{out}}) \times I_{\text{load}}$

Example from ProtoSupplies:

Using 9 V → 5 V at 1 A produces **4** W of heat.

A 7805 **definitely needs a heatsink** at that level.

☑ 2. How much can a 7805 dissipate without a heatsink?

Typical safe dissipation (TO-220 package):

- **~1–1.25 W without a heatsink** (depending on airflow and ambient temperature).
  This is confirmed by real-world measurements and forum engineering discussions, which note that around **1.25 W** is the upper limit before needing a sink.
  If your calculated power is **below ~1 W**, you are generally safe without a sink.

☑ 3. Examples

Case A: 9 V input, 5 V output, 200 mA load

$P = (9 - 5) \times 0.2 = 0.8$ W

➜ No heatsink needed; 0.8 W is within the 1 W limit.

Case B: 9 V input, 5 V output, 500 mA load

$P = 4 \times 0.5 = 2$ W

➜ Heatsink required (2 W > 1 W limit).

Case C: 12 V input, 5 V output, 300 mA load

$P = 7 \times 0.3 = 2.1$ W

➜ Heatsink definitely required.

☑ 4. When you DO need a heatsink

You need a heatsink if **any** of these are true:

- $(V_{in} - V_{out}) \times I_{load} > 1$ W
- The regulator becomes too hot to touch (above ~70 °C case)
- You are using it inside a closed plastic enclosure
- You draw ≥300 mA from 9–12 V
- You draw >100 mA from 12–15 V

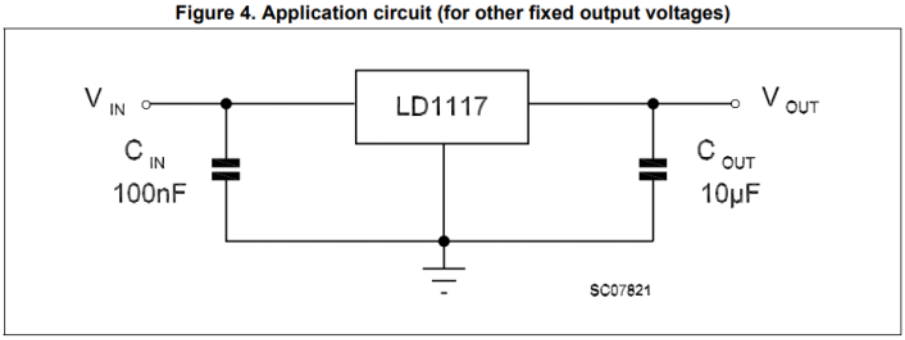
☞ If your load is:

- 100–250 mA from a 9 V supply → Probably not
- 300 mA or more from 9–12 V → Yes
- 500 mA or more → Definitely yes
- >1 A → Use a switching regulator instead; a heatsink alone won't be enough.

LD1117V33 3.3V regulator

Monday, 02 March 2026 21:17

Figure 4. Application circuit (for other fixed output voltages)



Notes on the 3.3 V Regulator (LD1117V33)

1. Purpose of the LD1117V33

The LD1117V33 is a **low-dropout (LDO) linear voltage regulator** designed to convert a higher input voltage down to a **regulated 3.3** V output. It can supply **up to 800** mA of load current. It belongs to the LD1117 family, which offers several fixed-output versions.

2. Key Features

- **Low dropout voltage**: Typically **1.0** V at 100 mA and up to **1.2** V at 800 mA.
- **Fixed 3.3 V output** option available (LD1117#33).
- **Output current up to 800 mA**.
- **Internal current limit** and **thermal shutdown** protection.
- **Requires only one output capacitor** for stability: **10** µF minimum.
- Tight output tolerance: **±1%** at 25 **°C.**

3. Input Voltage Requirements

- Maximum allowable input voltage: **15** V.
- Minimum input for regulated 3.3 V output: $V_{IN(min)} \approx 3.3\,\text{V} + \text{dropout}\ (\approx 1.1\,\text{V}) \approx 4.4\,\text{V}$
- Recommended to keep **VIN between 4.75** V and 10 V for best performance (datasheet test conditions).

4. Dropout Voltage

From the 3.3 V version's performance table:

Load current	Typical dropout	Max dropout
100 mA	1.0 V	1.1 V
500 mA	1.05 V	1.15 V
800 mA	1.10 V	1.2 V

Because of the significant dropout voltage, the LD1117 cannot regulate properly if the input is too close to 3.3 V.

5. Required Capacitors and Stability

Input capacitor

- The datasheet application circuit shows the input capacitor as optional, but usually included if the regulator is far from the supply or if the supply is noisy.

Output capacitor (mandatory)

- Minimum **10** µF capacitor required on the output for stable operation.
- Must have **low ESR** (common electrolytic or tantalum preferred).

This is different from the 7805, which needs **100** nF + 0.1–10 µF, while the LD1117 specifically depends on the ≈10 µF capacitor for loop stability.

6. Power Dissipation and Thermal Considerations

Like all linear regulators, the LD1117 dissipates power as heat:

$P = (V_{IN} - V_{OUT}) \times I_{OUT}$

The TO-220 package thermal resistance values:

- **RθJA = 50 °C/W** (junction-to-ambient)
- **RθJC = 5 °C/W** (junction-to-case)

Because RθJA is similar to the 7805's, the thermal guidelines are the same:

Heatsink needed if:

- Dissipation > ~1 W without heatsink
- Load > ~300 mA from 5 V or 9 V input
- Regulator runs hot to the touch

For example, with 9 V input:

$P = (9 - 3.3) \times 0.3A = 1.71\text{W} \Rightarrow \text{Heatsink required}$

7. Protection Features

The regulator includes:

- **Internal current limiting** to prevent overload damage.
- **Thermal shutdown**, which turns off the device if the die reaches excessive temperature.

These features prevent permanent failure but **do not** replace the need for proper heatsinking.

8. Comparison to 5 V (7805) Regulator

Feature	7805	LD1117V33
Dropout voltage	~2 V	~1.1 V
Max current	1–1.5 A	800 mA
Required caps	0.33 µF input, 0.1 µF output	**10 µF output mandatory**
Max VIN	35 V	15 V
Type	Standard linear regulator	Low-dropout linear regulator
Heat behaviour	Very similar	Very similar

The LD1117 is better for battery-powered or low-voltage input systems because of its lower dropout.

Light-Emitting Diodes

Monday, 02 March 2026 21:41

3_3V Regulator...

1. Demo 1 System Requirements for LEDs

According to the system specifications for Demo 1, you need to implement the following LED hardware:

- **Power Indicator (D6):** This LED must turn on to indicate that the system's 5V regulated voltage is present.
- **Status/Alarm Indicators (D2-D5):** The system uses a set of LEDs to act as visual alarm indicators for different warnings.
  - D2 acts as the proximity warning indicator.
  - D3 indicates unsafe driving and detected impacts.
  - D4 indicates low-light conditions.
  - D5 indicates uncomfortable temperatures.
- **Flashing Specifications:** The flashing timing for these status-indicating LEDs must be exactly 500 ms ON and 500 ms OFF.
- **Testing:** During Demo 1, the Test Station (TS) will visually monitor the power status LED (D6) and the status-indicating LEDs (D2-D5) to ensure they operate correctly on power-up.

2. Relevant Electrical Characteristics

To properly implement the Super Orange Red LED (NOR28L32) into your circuit, you need its electrical and optical characteristics at an ambient temperature of 25°C:

- **Forward Voltage (V_F):** The typical forward voltage is 2.0 V, with a minimum of 1.9 V and a maximum of 2.6 V.
- **Forward Current (I_F):** The recommended operating current is 20 mA.
- **Peak Forward Current:** It can handle a peak of 100 mA, tested at a 1/10 duty cycle at 1KHz.
- **Luminous Intensity (I_V):** The typical luminous intensity is 2500 mcd, with a maximum of 3000 mcd.
- **Reverse Voltage (V_R):** The absolute maximum reverse voltage the LED can withstand is 5 V.
- **Operating Temperature:** The LED is rated to operate safely between -40°C and 85°C.

3. Purpose of the Current-Limiting Resistor

- Because an LED is a current-operated device, it requires a current-limiting resistor to be incorporated into the drive circuit.
- This resistor drops the excess voltage from the supply and limits the current flowing through the LED to a safe level (such a s the recommended 20 mA).
- The manufacturer specifically recommends using a current-limiting resistor to ensure intensity uniformity when connecting LEDs in a circuit application.
- D6 is connected to the L78C5 (5V) regulator, $\therefore V_{supply} = 5V$
- From datasheet: $V_{F(TYP)} = 2V$, $I_{LED} = 0.02A$
  - *Normally use TYP values for calculations*
  - $\therefore R = \frac{5-2}{0.02} = 150\Omega$
- D2, D3, D4, D5 is connected to a GPIO pin with a voltage rating 3.3V *(pg. 38 from STM32f411RE datasheet)*
  - $\therefore R = \frac{3.3-2}{0.02} = 65\Omega \rightarrow 100\Omega$ closest approximate

Buttons

Monday, 02 March 2026 22:10

Button_Tac
tile_Switch

1. Demo 1 System Requirements for Buttons (SS8)

According to the system specifications for Demo 1, your tactile push-buttons (SS8) will be rigorously tested by the Test Station (TS):

- **Response Time:** The maximum allowable response time after any button push is exactly 500 ms. You must ensure your system registers and reacts to the input within this window.

2. Relevant Electrical & Mechanical Characteristics

To properly implement the tactile push-buttons, you should be familiar with the following component specifications from the manufacturer:

- **Maximum Rating:** The buttons can handle a maximum of DC 12 V and 50 mA.
- **Contact Resistance:** When closed, the contact resistance is 50 mΩ Max.
- **Insulation Resistance:** When open, the insulation resistance is 100 MΩ Min (tested at DC 100V).
- **Bounce Time:** The mechanical bounce when testing at "ON" and "OFF" is a maximum of 5 ms.
- **Actuating Force:** It takes a force of 250± 50 gf to trigger the button.
- **Travel Distance:** The physical travel of the button stem is 0.3 0.1± mm.
- **Operating Temperature:** The switches operate safely between -20°C and 70°C.

3. Hardware Implementation: Resistors and Debouncing

When integrating these buttons into your STM32 microcontroller circuit, you must account for two major hardware factors:

- **Pull-up / Pull-down Resistors:** When a tactile switch is unpressed (open), the microcontroller's GPIO pin is physically disconnected and left "floating." This will cause unpredictable logic states (picking up random electrical noise). To fix this, you must implement a **pull-up resistor** (tying the pin to your 3.3V logic high) or a **pull-down resistor** (tying the pin to ground). This ensures the pin sits at a known state until the button is pressed to pull it to the opposite state. *(Note: The STM32 has internal pull-up/pull-down resistors that you can enable in software, but you must know how to implement them in hardware if asked).*
- **Hardware Debouncing:** Because the spec sheet explicitly states the button has a bounce time of up to 5 ms, pressing it once will cause the electrical signal to rapidly spike between HIGH and LOW for a few milliseconds before settling. If unaccounted for, the microcontroller will register a single push as multiple rapid pushes. While you can handle this in software (by adding a >5ms delay after an initial trigger), you can also implement **hardware debouncing** using a simple RC (Resistor-Capacitor) low-pass filter circuit to smooth out the fluctuating signal.

LMT01 Temperature Sensor

Monday, 02 March 2026 22:21

LMT01 Temperat...

1. Demo 1 System Requirements for the Temperature Sensor

According to the system specifications for Demo 1:

- **Testing & Verification:** The Test Station (TS) will monitor and test the output of the digital temperature sensor via UART.
- **UART Communication:** The measured temperature data must be transmitted using the clearly specified Demonstration 1 UART message format.

2. Relevant Electrical Characteristics (LMT01)

To successfully implement the LMT01 temperature sensor in your hardware, you must understand its unique operating parameters:

- **Sensor Type:** The LMT01 is a highly accurate, 2-pin digital output temperature sensor.
- **Current Loop Interface:** It utilizes a pulse count current loop interface that is easily read by a processor. The number of output pulses it transmits is proportional to the temperature, offering a 0.0625℃ resolution.
- **Accuracy:** It maintains a maximum accuracy of ±0.5℃ between -20℃ and 90℃.
- **Base Current:** During operation, it draws a conversion current of 34μA.
- **Frequency & Period:** The communication frequency of the sensor is 88 kHz. The continuous conversion plus data-transmission period is 100 ms.
- **Supply:** The sensor features a floating 2-V to 5.5-V (VP-VN) supply operation with integrated EMI immunity.

3. Hardware Implementation Details

- **Microcontroller Interface:** The LMT01's pulse count interface can be connected to the microcontroller using either a GPIO pin or a Comparator (COMP) input.

OUTPUT CURRENT						
I_{OL}	Output current variation	Low level	28	34	40	µA
I_{OH}		High level	112.5	125	143	µA
	High-to-Low level output current ratio		3.1	3.7	4.5	

  - **Current Loop to Voltage Conversion:** The LMT01 temperature sensor transmits its data using a pulse count *current loop* interface, meaning it communicates by modulating the amount of current it draws. However, the microcontroller (STM32) reads *voltage* changes on its GPIO or Comparator (COMP) pins.
  - **Applying Ohm's Law:** By placing a pull-down resistor between the sensor's VN terminal and ground, the current flowing out of the sensor is forced to pass through this resistor. According to Ohm's Law, this current creates a voltage drop across the resistor.
    - For $(v_o)_{STM} = 0.99V$: $R_H = \frac{0.99}{34\times10^{-6}} = 29.12k\Omega$
    - For $(v_o)_{STM} = 2.31V$: $R_H = \frac{2.31}{125\times10^{-6}} = 18.48k\Omega$
  - **Creating Readable Pulses:** When the LMT01 pulses its current high and low to transmit temperature data, the pull-down resistor converts those fluctuating currents into corresponding high and low *voltage* pulses. This allows the microcontroller to easily read and count the pulses to determine the temperature.
- **Timing Constraints (Lecture Advice):** The lecture slides explicitly highlight the strict timing windows you must account for when reading the pulses:
  - There is a **50 ms** window between the start and the end of the data transmission.
  - There is a **104 ms** interval between the start of one data transmission until the start of the next data transmission.
- **Data Handling:** You should calculate an accurate mean and avoid anomalies when processing the readings.

UART Student number

Monday, 02 March 2026 22:41

```c
void SendStudent_Number(void)
{
    HAL_Delay(350);
    char uart_buffer[20];
    const char studentNumber[] = "*24989142#\n";
    sprintf(uart_buffer, "%s", studentNumber);
    HAL_UART_Transmit(&huart2, (uint8_t*)studentNumber, strlen(studentNumber), 500);
}
```

1. UART Communication: Analysis and Implementation

The core software task for Demo 1 is managing UART communication on the USART2 peripheral. This involves sending a startup message and processing an incoming command.

1.1. UART Initialization (`MX_USART2_UART_Init`)

The USART2 peripheral must be configured with parameters that match the Test Station's expectations. The initialization code sets up the following:

```c
huart2.Instance = USART2;
huart2.Init.BaudRate = 57600;
huart2.Init.WordLength = UART_WORDLENGTH_9B;  // 8 data bits + 1 parity bit
huart2.Init.StopBits = UART_STOPBITS_1;
huart2.Init.Parity = UART_PARITY_EVEN;
huart2.Init.Mode = UART_MODE_TX_RX;
huart2.Init.HwFlowCtl = UART_HWCONTROL_NONE;
huart2.Init.OverSampling = UART_OVERSAMPLING_16;
huart2.Init.OneBitSampling = UART_ONE_BIT_SAMPLE_DISABLE;
huart2.Init.ClockPrescaler = UART_PRESCALER_DIV1;
huart2.AdvancedInit.AdvFeatureInit = UART_ADVFEATURE_NO_INIT;
```

- **Insight:** This configuration is critical. Any terminal program used for testing must be set to **57600 baud, 8 data bits, Even parity, and 1 stop bit**. The `UART_WORDLENGTH_9B` setting is how the STM32 HAL accommodates the parity bit alongside the data bits

1.2. Transmitting the Student Number on Startup (SS2)

The `SendStudent_Number` function is called once after initialization to meet the first requirement of Demo 1.

```c
void SendStudent_Number(void) {
    HAL_Delay(150);
    char uart_buffer[20];
    const char studentNumber[] = "*24989142#\n";
    sprintf(uart_buffer, "%s", studentNumber);
    HAL_UART_Transmit(&huart2, (uint8_t*)studentNumber, strlen(studentNumber), 500);
}
```

- **Transmission Time Calculation:**
  - Each character frame consists of 1 start bit + 8 data bits + 1 parity bit + 1 stop bit = 10 bits
  - The string `*24989142#\n` is 11 characters long
  - Total bits to transmit: $11 \text{ characters} \times 10 \frac{\text{bits}}{\text{character}} = 110 \text{ bits}$
  - Time to transmit at 57600 bps: $\frac{110 \text{ bits}}{57600 \text{ bits/sec}} \approx 1.91\ ms$. This is extremely fast and well within the 500 ms timesout

LED flashing

Monday, 02 March 2026 22:42

```
void UpdateLEDs(void)
{
    uint32_t now = HAL_GetTick();

    // 2 Hz flash = 500ms ON, 500ms OFF
    if ((now - lastFlashTime) >= 500)
    {
        lastFlashTime = now;
        flashToggle ^= 1;

        // D3 = PB4, flashes when state is ON
        if (led_d3_state)
            HAL_GPIO_WritePin(GPIOB, GPIO_PIN_4, flashToggle ? GPIO_PIN_SET : GPIO_PIN_RESET);
        else
            HAL_GPIO_WritePin(GPIOB, GPIO_PIN_4, GPIO_PIN_RESET);

        // D5 = PC4, flashes when state is ON
        if (led_d5_state)
            HAL_GPIO_WritePin(GPIOC, GPIO_PIN_4, flashToggle ? GPIO_PIN_SET : GPIO_PIN_RESET);
        else
            HAL_GPIO_WritePin(GPIOC, GPIO_PIN_4, GPIO_PIN_RESET);
    }

    // D2 = PB5, static ON/OFF
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, led_d2_state ? GPIO_PIN_SET : GPIO_PIN_RESET);

    // D4 = PA10, static ON/OFF
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10,  led_d4_state ? GPIO_PIN_SET : GPIO_PIN_RESET);
}
```

Button-pressing and Debouncing

Monday, 02 March 2026 22:44

```c
void S1_Button(void)
{
    static uint8_t state = 0;
    static uint32_t press_time = 0;

    uint8_t current = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0);
    if (current == 1 && state == 0)
    {
        // Button just pressed
        press_time = HAL_GetTick();
        state = 1;
    }
    if (current == 0 && state == 1)
    {
        // Button released
        uint32_t duration = HAL_GetTick() - press_time;
        if (duration > 100)   // must be held 100ms
        {
            led_d2_state = !led_d2_state;
        }
        state = 0;
    }
}
```

```c
void S3_Button(void)
{
    static uint8_t state = 0;
    static uint32_t press_time = 0;

    uint8_t current = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_7);

    if (current == 1 && state == 0)
    {
        press_time = HAL_GetTick();
        state = 1;
    }

    if (current == 0 && state == 1)
    {
        uint32_t duration = HAL_GetTick() - press_time;

        if (duration > 100)
        {
            char msg[50];

            if (temp_ready)
                sprintf(msg, "@%.1f&\r\n", current_temp);
            else
                sprintf(msg, "Reading sensor...\r\n");

            HAL_UART_Transmit(&huart2, (uint8_t*)msg, strlen(msg), 100);
        }

        state = 0;
    }
}
```

Temperature Sensor

Monday, 02 March 2026 22:45

```c
void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim)
{
    if ((htim->Instance == TIM3) && (htim->Channel == HAL_TIM_ACTIVE_CHANNEL_2))
    {
        uint32_t capture = HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_2);

        uint32_t diff;
        if (capture >= last_capture_time)
            diff = capture - last_capture_time;
        else
            diff = (0xFFFF - last_capture_time + capture + 1);

        last_capture_time = capture;

        if (diff != 0)
        {
            float timer_clock = HAL_RCC_GetPCLK1Freq() * 2.0f;
            float frequency = timer_clock / diff;

            // ✅ Correct conversion
            float temperature = (frequency / 4096.0f) - 50.0f;

            // ✅ Filter AFTER conversion
            current_temp = CalibrateTemperature(temperature);

            temp_ready = 1;
        }
    }
}
```

```c
float CalibrateTemperature(float new_temp)
{
    temp_buffer[temp_index++] = new_temp;

    if (temp_index >= TEMP_BUFFER_SIZE)
    {
        temp_index = 0;
        buffer_full = 1;
    }

    if (!buffer_full) return new_temp;

    // Copy buffer
    float temp_copy[5];
    memcpy(temp_copy, temp_buffer, sizeof(temp_buffer));

    // Sort (simple bubble sort)
    for (int i = 0; i < 4; i++)
        for (int j = i + 1; j < 5; j++)
            if (temp_copy[i] > temp_copy[j])
            {
                float t = temp_copy[i];
                temp_copy[i] = temp_copy[j];
                temp_copy[j] = t;
            }

    // Average middle 3 values
    return (temp_copy[1] + temp_copy[2] + temp_copy[3]) / 3.0f;
}
```

Demo 1 Software Implementation

Monday, 02 March 2026 22:24

```
/* USER CODE BEGIN Header */
/**
  ******************************************************************************
  * @file           : main.c
  * @brief          : Main program body
  ******************************************************************************
  * @attention
  *
  * Copyright (c) 2026 STMicroelectronics.
  * All rights reserved.
  *
  * This software is licensed under terms that can be found in the LICENSE file
  * in the root directory of this software component.
  * If no LICENSE file comes with this software, it is provided AS-IS.
  *
  ******************************************************************************
  */
/* USER CODE END Header */
/* Includes ------------------------------------------------------------------*/
#include "main.h"

/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */
#include <string.h>
#include <stdio.h>

/* USER CODE END Includes */

/* Private typedef -----------------------------------------------------------*/
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define ------------------------------------------------------------*/
/* USER CODE BEGIN PD */
#define BUTTON_DEBOUNCE_TIME_MS 200
#define TEMP_BUFFER_SIZE 5

/* USER CODE END PD */

/* Private macro -------------------------------------------------------------*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables ---------------------------------------------------------*/
TIM_HandleTypeDef htim3;

UART_HandleTypeDef huart2;

/* USER CODE BEGIN PV */
// LED state tracking
static uint8_t led_d2_state = 1;  // D2 starts ON (continuously)
uint8_t led_d3_state = 1;  // D3 starts flashing
static uint8_t led_d4_state = 1;  // D4 starts ON (continuously)
uint8_t led_d5_state = 1;  // D5 starts flashing

// Flash timing
uint32_t lastFlashTime = 0;
```

```c
uint8_t flashToggle = 0;

// Debounce trackers for each specific button
uint32_t LastTick_S1_PA0  = 0; // S1
uint32_t LastTick_S2_PB8  = 0; // S2
uint32_t LastTick_S3_PA7  = 0; // S3
uint32_t LastTick_S4_PB6  = 0; // S4
uint32_t LastTick_S5_PB9  = 0; // S5

// Temperature sensor tracking
volatile uint32_t pulse_count = 0;
volatile uint32_t last_capture_time = 0;
volatile float current_temp = -50.0f;
volatile uint8_t temp_ready = 0;

// Temperature sensor calibration
float temp_buffer[TEMP_BUFFER_SIZE] = {0};
uint8_t temp_index = 0;
uint8_t buffer_full = 0;

/* USER CODE END PV */

/* Private function prototypes -----------------------------------------------*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_USART2_UART_Init(void);
static void MX_TIM3_Init(void);
/* USER CODE BEGIN PFP */
float CalibrateTemperature(float new_temp);

/* USER CODE END PFP */

/* Private user code ---------------------------------------------------------*/
/* USER CODE BEGIN 0 */
void SendStudent_Number(void)
{
    HAL_Delay(350);
    char uart_buffer[20];
    const char studentNumber[] = "*24989142#\n";
    sprintf(uart_buffer, "%s", studentNumber);
    HAL_UART_Transmit(&huart2, (uint8_t*)studentNumber, strlen(studentNumber), 500);
}

void UpdateLEDs(void)
{
    uint32_t now = HAL_GetTick();

    // 2 Hz flash = 500ms ON, 500ms OFF
    if ((now - lastFlashTime) >= 500)
    {
        lastFlashTime = now;
        flashToggle ^= 1;

        // D3 = PB4, flashes when state is ON
        if (led_d3_state)
            HAL_GPIO_WritePin(GPIOB, GPIO_PIN_4, flashToggle ? GPIO_PIN_SET : GPIO_PIN_RESET);
        else
            HAL_GPIO_WritePin(GPIOB, GPIO_PIN_4, GPIO_PIN_RESET);

        // D5 = PC4, flashes when state is ON
        if (led_d5_state)
            HAL_GPIO_WritePin(GPIOC, GPIO_PIN_4, flashToggle ? GPIO_PIN_SET : GPIO_PIN_RESET);
        else
            HAL_GPIO_WritePin(GPIOC, GPIO_PIN_4, GPIO_PIN_RESET);
```

```
    }

    // D2 = PB5, static ON/OFF
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, led_d2_state ? GPIO_PIN_SET : GPIO_PIN_RESET);

    // D4 = PA10, static ON/OFF
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10,  led_d4_state ? GPIO_PIN_SET : GPIO_PIN_RESET);
}

void S1_Button(void)
{
    static uint8_t state = 0;
    static uint32_t press_time = 0;

    uint8_t current = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0);
    if (current == 1 && state == 0)
    {
        // Button just pressed
        press_time = HAL_GetTick();
        state = 1;
    }
    if (current == 0 && state == 1)
    {
        // Button released
        uint32_t duration = HAL_GetTick() - press_time;
        if (duration > 100)   // must be held 100ms
        {
            led_d2_state = !led_d2_state;
        }
        state = 0;
    }
}

void S2_Button(void)
{
    static uint8_t state = 0;
    static uint32_t press_time = 0;

    uint8_t current = HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_8);

    if (current == 1 && state == 0)
    {
        // Button just pressed
        press_time = HAL_GetTick();
        state = 1;
    }
    if (current == 0 && state == 1)
    {
        // Button released
        uint32_t duration = HAL_GetTick() - press_time;
        if (duration > 100)   // must be held 100ms
        {
            led_d3_state = !led_d3_state;
        }
        state = 0;
    }
}

void S3_Button(void)
{
    static uint8_t state = 0;
    static uint32_t press_time = 0;

    uint8_t current = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_7);
```

```
    if (current == 1 && state == 0)
    {
        press_time = HAL_GetTick();
        state = 1;
    }

    if (current == 0 && state == 1)
    {
        uint32_t duration = HAL_GetTick() - press_time;

        if (duration > 100)
        {
            char msg[50];

            if (temp_ready)
                sprintf(msg, "@%.1f&\r\n", current_temp);
            else
                sprintf(msg, "Reading sensor...\r\n");

            HAL_UART_Transmit(&huart2, (uint8_t*)msg, strlen(msg), 100);
        }

        state = 0;
    }
}

void S4_Button(void)
{
    static uint8_t state = 0;
    static uint32_t press_time = 0;

    uint8_t current = HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_6);

    if (current == 1 && state == 0)
    {
        // Button just pressed
        press_time = HAL_GetTick();
        state = 1;
    }
    if (current == 0 && state == 1)
    {
        // Button released
        uint32_t duration = HAL_GetTick() - press_time;
        if (duration > 100)   // must be held 100ms
        {
            led_d4_state = !led_d4_state;
        }
        state = 0;
    }
}

void S5_Button(void)
{
    static uint8_t state = 0;
    static uint32_t press_time = 0;

    uint8_t current = HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_9);

    if (current == 1 && state == 0)
    {
        // Button just pressed
        press_time = HAL_GetTick();
        state = 1;
```

```
    }
    if (current == 0 && state == 1)
    {
        // Button released
        uint32_t duration = HAL_GetTick() - press_time;
        if (duration > 100)   // must be held 100ms
        {
            led_d5_state = !led_d5_state;
        }
        state = 0;
    }
}

/* USER CODE END 0 */

/**
  * @brief  The application entry point.
  * @retval int
  */
int main(void)
{

  /* USER CODE BEGIN 1 */

  /* USER CODE END 1 */

  /* MCU Configuration--------------------------------------------------------*/

  /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
  HAL_Init();

  /* USER CODE BEGIN Init */

  /* USER CODE END Init */

  /* Configure the system clock */
  SystemClock_Config();

  /* USER CODE BEGIN SysInit */

  /* USER CODE END SysInit */

  /* Initialize all configured peripherals */
  MX_GPIO_Init();
  MX_USART2_UART_Init();
  MX_TIM3_Init();
  /* USER CODE BEGIN 2 */
  SendStudent_Number();

  HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, GPIO_PIN_SET);    // D2 ON
  HAL_GPIO_WritePin(GPIOB, GPIO_PIN_4,  GPIO_PIN_SET);   // D3 starts ON
  HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10,  GPIO_PIN_SET);  // D4 ON
  HAL_GPIO_WritePin(GPIOC, GPIO_PIN_4,  GPIO_PIN_SET);   // D5 starts ON

  flashToggle = 1;
  lastFlashTime = HAL_GetTick();

  // Start TIM3 input capture in interrupt mode
  HAL_TIM_IC_Start_IT(&htim3, TIM_CHANNEL_2);

  /* USER CODE END 2 */

  /* Infinite loop */
```

```
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
      UpdateLEDs();
      S1_Button();
      S2_Button();
      S3_Button();  // sends current_temperature on press
      S4_Button();
      S5_Button();
  }
  /* USER CODE END 3 */
}

/**
  * @brief System Clock Configuration
  * @retval None
  */
void SystemClock_Config(void)
{
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

  /** Configure the main internal regulator output voltage
  */
  __HAL_RCC_PWR_CLK_ENABLE();
  __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE1);

  /** Initializes the RCC Oscillators according to the specified parameters
  * in the RCC_OscInitTypeDef structure.
  */
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
  RCC_OscInitStruct.HSIState = RCC_HSI_ON;
  RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
  RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSI;
  RCC_OscInitStruct.PLL.PLLM = 16;
  RCC_OscInitStruct.PLL.PLLN = 336;
  RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV4;
  RCC_OscInitStruct.PLL.PLLQ = 4;
  if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
  {
    Error_Handler();
  }

  /** Initializes the CPU, AHB and APB buses clocks
  */
  RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                              |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

  if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) != HAL_OK)
  {
    Error_Handler();
  }
}

/**
  * @brief TIM3 Initialization Function
```

```
  * @param None
  * @retval None
  */
static void MX_TIM3_Init(void)
{

  /* USER CODE BEGIN TIM3_Init 0 */

  /* USER CODE END TIM3_Init 0 */

  TIM_MasterConfigTypeDef sMasterConfig = {0};
  TIM_IC_InitTypeDef sConfigIC = {0};

  /* USER CODE BEGIN TIM3_Init 1 */

  /* USER CODE END TIM3_Init 1 */
  htim3.Instance = TIM3;
  htim3.Init.Prescaler = 0;
  htim3.Init.CounterMode = TIM_COUNTERMODE_UP;
  htim3.Init.Period = 65535;
  htim3.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
  htim3.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
  if (HAL_TIM_IC_Init(&htim3) != HAL_OK)
  {
    Error_Handler();
  }
  sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
  sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
  if (HAL_TIMEx_MasterConfigSynchronization(&htim3, &sMasterConfig) != HAL_OK)
  {
    Error_Handler();
  }
  sConfigIC.ICPolarity = TIM_INPUTCHANNELPOLARITY_RISING;
  sConfigIC.ICSelection = TIM_ICSELECTION_DIRECTTI;
  sConfigIC.ICPrescaler = TIM_ICPSC_DIV1;
  sConfigIC.ICFilter = 0;
  if (HAL_TIM_IC_ConfigChannel(&htim3, &sConfigIC, TIM_CHANNEL_2) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN TIM3_Init 2 */

  /* USER CODE END TIM3_Init 2 */

}

/**
  * @brief USART2 Initialization Function
  * @param None
  * @retval None
  */
static void MX_USART2_UART_Init(void)
{

  /* USER CODE BEGIN USART2_Init 0 */

  /* USER CODE END USART2_Init 0 */

  /* USER CODE BEGIN USART2_Init 1 */

  /* USER CODE END USART2_Init 1 */
  huart2.Instance = USART2;
  huart2.Init.BaudRate = 57600;
  huart2.Init.WordLength = UART_WORDLENGTH_9B;
```

```
  huart2.Init.StopBits = UART_STOPBITS_1;
  huart2.Init.Parity = UART_PARITY_EVEN;
  huart2.Init.Mode = UART_MODE_TX_RX;
  huart2.Init.HwFlowCtl = UART_HWCONTROL_NONE;
  huart2.Init.OverSampling = UART_OVERSAMPLING_16;
  if (HAL_UART_Init(&huart2) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN USART2_Init 2 */

  /* USER CODE END USART2_Init 2 */

}

/**
  * @brief GPIO Initialization Function
  * @param None
  * @retval None
  */
static void MX_GPIO_Init(void)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};
  /* USER CODE BEGIN MX_GPIO_Init_1 */

  /* USER CODE END MX_GPIO_Init_1 */

  /* GPIO Ports Clock Enable */
  __HAL_RCC_GPIOC_CLK_ENABLE();
  __HAL_RCC_GPIOH_CLK_ENABLE();
  __HAL_RCC_GPIOA_CLK_ENABLE();
  __HAL_RCC_GPIOB_CLK_ENABLE();

  /*Configure GPIO pin Output Level */
  HAL_GPIO_WritePin(GPIOA, LD2_Pin|GPIO_PIN_10, GPIO_PIN_RESET);

  /*Configure GPIO pin Output Level */
  HAL_GPIO_WritePin(GPIOC, GPIO_PIN_4, GPIO_PIN_RESET);

  /*Configure GPIO pin Output Level */
  HAL_GPIO_WritePin(GPIOB, GPIO_PIN_4|GPIO_PIN_5, GPIO_PIN_RESET);

  /*Configure GPIO pin : B1_Pin */
  GPIO_InitStruct.Pin = B1_Pin;
  GPIO_InitStruct.Mode = GPIO_MODE_IT_FALLING;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  HAL_GPIO_Init(B1_GPIO_Port, &GPIO_InitStruct);

  /*Configure GPIO pins : PA0 PA7 */
  GPIO_InitStruct.Pin = GPIO_PIN_0|GPIO_PIN_7;
  GPIO_InitStruct.Mode = GPIO_MODE_IT_RISING;
  GPIO_InitStruct.Pull = GPIO_PULLDOWN;
  HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

  /*Configure GPIO pins : LD2_Pin PA10 */
  GPIO_InitStruct.Pin = LD2_Pin|GPIO_PIN_10;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

  /*Configure GPIO pin : PC4 */
  GPIO_InitStruct.Pin = GPIO_PIN_4;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
```

```
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);

  /*Configure GPIO pins : PB4 PB5 */
  GPIO_InitStruct.Pin = GPIO_PIN_4|GPIO_PIN_5;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

  /*Configure GPIO pins : PB6 PB8 PB9 */
  GPIO_InitStruct.Pin = GPIO_PIN_6|GPIO_PIN_8|GPIO_PIN_9;
  GPIO_InitStruct.Mode = GPIO_MODE_IT_RISING;
  GPIO_InitStruct.Pull = GPIO_PULLDOWN;
  HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

  /* USER CODE BEGIN MX_GPIO_Init_2 */

  /* USER CODE END MX_GPIO_Init_2 */
}

/* USER CODE BEGIN 4 */
float CalibrateTemperature(float new_temp)
{
    temp_buffer[temp_index++] = new_temp;

    if (temp_index >= TEMP_BUFFER_SIZE)
    {
        temp_index = 0;
        buffer_full = 1;
    }

    if (!buffer_full) return new_temp;

    // Copy buffer
    float temp_copy[5];
    memcpy(temp_copy, temp_buffer, sizeof(temp_buffer));

    // Sort (simple bubble sort)
    for (int i = 0; i < 4; i++)
        for (int j = i + 1; j < 5; j++)
            if (temp_copy[i] > temp_copy[j])
            {
                float t = temp_copy[i];
                temp_copy[i] = temp_copy[j];
                temp_copy[j] = t;
            }

    // Average middle 3 values
    return (temp_copy[1] + temp_copy[2] + temp_copy[3]) / 3.0f;
}

void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim)
{
    if ((htim->Instance == TIM3) && (htim->Channel == HAL_TIM_ACTIVE_CHANNEL_2))
    {
        uint32_t capture = HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_2);

        uint32_t diff;
        if (capture >= last_capture_time)
            diff = capture - last_capture_time;
        else
            diff = (0xFFFF - last_capture_time + capture + 1);
```

```
        last_capture_time = capture;

        if (diff != 0)
        {
             float timer_clock = HAL_RCC_GetPCLK1Freq() * 2.0f;
            float frequency = timer_clock / diff;

            // ✅ Correct conversion
            float temperature = (frequency / 4096.0f) - 50.0f;

            // ✅ Filter AFTER conversion
            current_temp = CalibrateTemperature(temperature);

            temp_ready = 1;
        }
    }
}


/* USER CODE END 4 */

/**
  * @brief  This function is executed in case of error occurrence.
  * @retval None
  */
void Error_Handler(void)
{
  /* USER CODE BEGIN Error_Handler_Debug */
  /* User can add his own implementation to report the HAL error return state */
  __disable_irq();
  while (1)
  {
  }
  /* USER CODE END Error_Handler_Debug */
}
#ifdef USE_FULL_ASSERT
/**
  * @brief  Reports the name of the source file and the source line number
  *         where the assert_param error has occurred.
  * @param  file: pointer to the source file name
  * @param  line: assert_param error line source number
  * @retval None
  */
void assert_failed(uint8_t *file, uint32_t line)
{
  /* USER CODE BEGIN 6 */
  /* User can add his own implementation to report the file name and line number,
     ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
  /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */
```